Project Report

Winter Semester 2025-26

Computer Vision (CSE 3010)

SLOT – D11+D12

Project Type: Bring Your Own Project (BYOP)

Submitted by:

Name: Aarya Butolia

Reg. No: 23BAI10414

Submitted to:

Faculty: Dr. Rajneesh Patel

Vityarthi Project

1. Introduction

Object detection is an important area of Computer Vision that involves identifying and locating objects within images or video streams. Unlike image classification, which only predicts the category of an image, object detection provides both the class label and the spatial location (bounding box) of objects.

This project presents a real-time object detection system using **YOLOv8 (You Only Look Once – Version 8)**. The system is capable of detecting multiple objects across different input sources such as images, videos, and live webcam feeds.

The main objective of this project is to develop an efficient, accurate, and real-time object detection pipeline that can be used in practical applications such as surveillance, automation, and smart monitoring systems.

---

2. Problem Statement

Manual detection and identification of objects in images and videos is time-consuming and inefficient. With increasing data and real-time requirements, there is a need for an automated system.

This project aims to solve the following problems:

1. Detect multiple objects in real-time with high accuracy
2. Process different types of inputs (images, videos, webcam)

3. Provide visual outputs with bounding boxes and confidence scores
4. Generate analytical insights from detection results

---

3. Objectives

- To implement real-time object detection using YOLOv8
- To support image, video, and webcam input
- To visualize detections using bounding boxes
- To analyze detection results using graphs
- To build a user-friendly and efficient system

---

4. Methodology

The system uses a deep learning-based object detection algorithm called **YOLOv8**, which is known for its speed and accuracy.

Working Principle:

- **Input Processing:**
  The system takes input in the form of an image, video frame, or webcam stream.
- **YOLOv8 Model:**
  A pre-trained model (yolov8n.pt) trained on the COCO dataset is used.

- **Single-Shot Detection:**
  YOLO processes the entire image in a single pass, making it extremely fast.
- **Object Detection:**
  The model predicts:
  - Object class
  - Confidence score
  - Bounding box coordinates
- **Filtering:**
  Detections below a defined confidence threshold are ignored.
- **Visualization:**
  Bounding boxes and labels are drawn on the detected objects.
- **Analysis:**
  Detection data is analyzed to generate statistics and graphs.

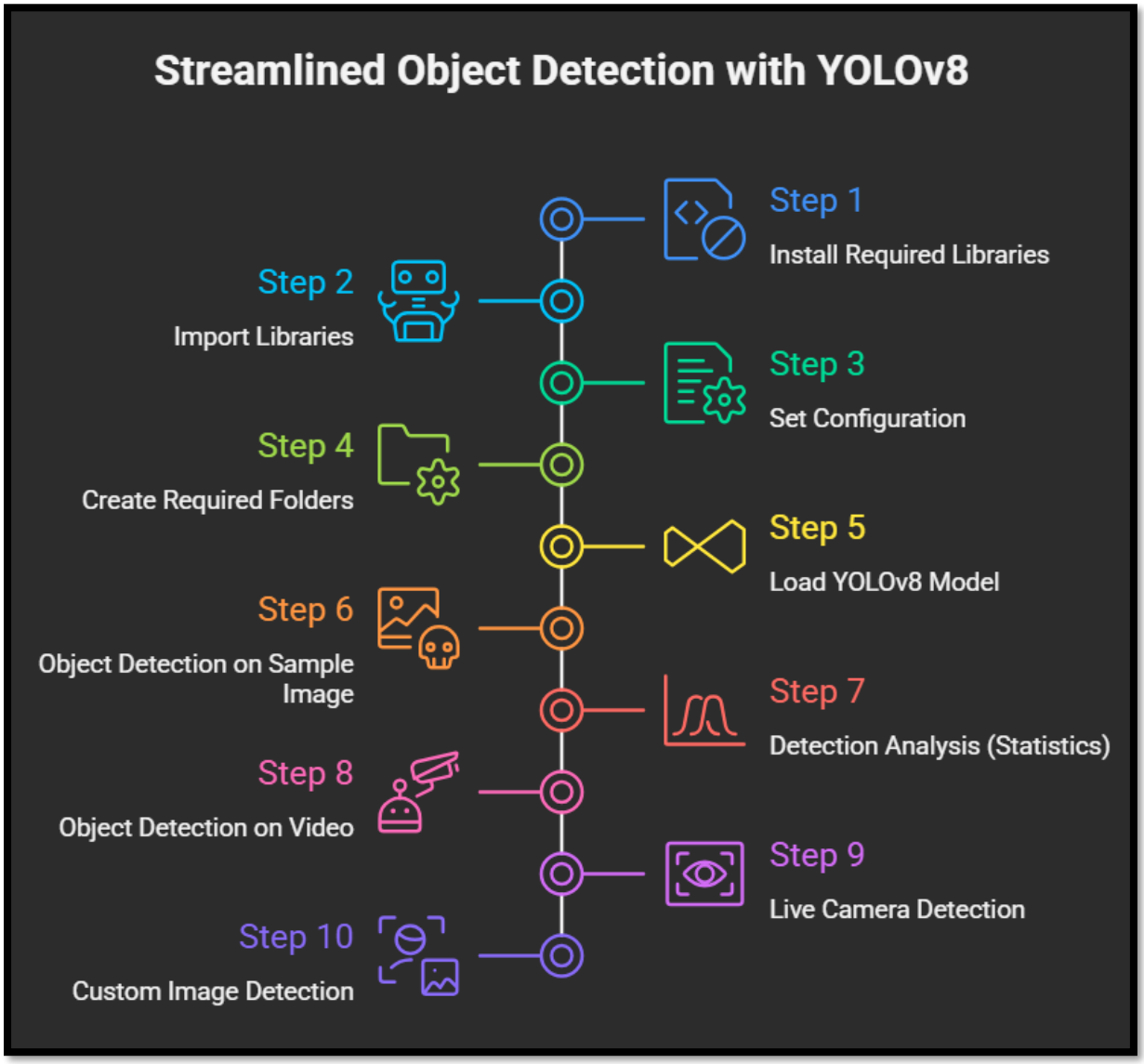
---

5. Workflow

Step-by-Step Workflow:

1. Install required libraries (Ultralytics, OpenCV, Matplotlib, etc.)
2. Load the YOLOv8 pre-trained model
3. Provide input (image/video/webcam)

4. Process input data
5. Run YOLOv8 detection
6. Extract object details (class, confidence, bounding box)
7. Draw bounding boxes on detected objects
8. Save annotated output
9. Generate statistical analysis (graphs)
10. Display final results



6. System Design & Implementation

The system is implemented using **Python** and follows a modular approach.

Technologies Used:

- Python 3.x
- OpenCV
- Ultralytics YOLOv8
- Matplotlib
- NumPy

Modules:

1. Image Detection Module

- Detects objects in static images
- Saves annotated images

2. Video Detection Module

- Processes video frame-by-frame
- Saves output video

3. Webcam Detection Module

- Captures live video
- Allows real-time detection
- Supports snapshot detection

4. Analytics Module

- Generates:
  - Bar chart (object count)
  - Histogram (confidence scores)

---

7. Results and Discussion

Performance:

- Real-time detection achieved (~20–30 FPS depending on system)
- Works efficiently on CPU

Accuracy:

- High accuracy for common objects
- Best performance in good lighting conditions

Outputs Generated:

- Annotated images
- Processed video output
- Detection statistics graphs

Limitations:

- Reduced accuracy in low-light conditions
- Performance decreases for very small or occluded objects
- Heavy models may require GPU for faster processing

---

8. Advantages

- Fast and real-time detection
- Supports multiple input types
- Easy to use and implement
- Lightweight model available (YOLOv8n)

---

9. Applications

- Smart surveillance systems
- Traffic monitoring
- Autonomous vehicles
- Industrial automation
- Security systems

---

10. Conclusion

The project successfully demonstrates real-time object detection using YOLOv8. It highlights the efficiency and speed of modern deep learning-based detection techniques.

Compared to traditional methods, YOLOv8 provides better accuracy and performance. The system is scalable and can be extended for advanced AI-based applications.

---

11. Future Scope

- Integration with cloud systems

- Use of custom-trained models
- Deployment on mobile or edge devices
- Integration with alert systems

---

12. References

1. Ultralytics YOLOv8 Documentation
   https://docs.ultralytics.com/
2. COCO Dataset
   https://cocodataset.org/
3. OpenCV Documentation
   https://docs.opencv.org/
4. Python Documentation
   https://www.python.org/

---

13. Output Samples

YOLOv8 Object Detection — 4 object(s) found | conf >= 35%